

Projeto Prático #1: O projeto deve ser desenvolvido por grupos de *no máximo* três alunos. É estritamente proibida a partilha de código entre alunos de grupos diferentes.

Plágio: Além de inspeção manual, será também executado um software de deteção de plágio no código-fonte do projeto. *Todos* os alunos que submeterem código obtido através de plágio terão os seus projetos anulados.

Uso de Inteligência Artificial: É permitido utilizar ferramentas de IA como o ChatGPT, Copilot, DeepSeek e similares. No entanto, todos os membros do grupo devem ser capazes de compreender e *explicar* o projeto *na sua totalidade* aos docentes quando pedido. Se houver código no projeto que o grupo não consiga explicar, o projeto será considerado como *plágio*.

Submissão: A data de submissão do projeto é **03/11/2025** às 23:59. A descrição do processo de submissão pode ser encontrada no enunciado.

WeatherWise: Serviço de Meteorologia

1 Contextualização

A empresa **ConcreteCast Solutions (CCS¹)** identificou uma oportunidade de mercado para fornecer dados meteorológicos através de uma API simples e robusta. A ideia é criar um serviço autónomo, chamado **WeatherWise**, que possa ser consumido por outras aplicações, como sistemas de gestão de transportes públicos, aplicações de eventos ou plataformas de turismo.

O serviço deve ser capaz de fornecer a previsão do tempo atual para uma determinada localidade. Além disso, para permitir análises e planeamento, o WeatherWise deve manter um registo histórico das condições climáticas diárias e ser capaz de auditar todas as consultas que recebe.

Como funcionalidade diferenciadora, o serviço deverá oferecer uma capacidade de estimação: com base nos dados históricos de uma localidade, o sistema deve ser capaz de calcular uma previsão provável para uma data futura. Esta funcionalidade será particularmente útil para clientes que necessitem de fazer planeamentos a médio prazo sem recorrer a modelos meteorológicos complexos e dispendiosos.

O objetivo desta primeira fase é construir este serviço como um sistema completamente isolado, focando-se na lógica de negócio e na interação com a base de dados, utilizando uma arquitetura simples e de base.

2 Casos de Uso

Nesta primeira fase, o objetivo é implementar a totalidade do serviço WeatherWise. Apresenta-se a lista completa dos casos de uso para o serviço WeatherWise, que devem ser **todos** implementados.

- A. **Consultar previsão atual:** Dado o nome de uma localidade, o sistema devolve a condição climática atual registada (ex: “Sol”, “Chuva”, “Trovoadas”) e a temperatura.

¹Entidade fictícia

- B. **Atualizar/criar previsão histórica:** Permite inserir ou atualizar a condição climática para uma localidade numa data e localidade específica.
- C. **Obter histórico de auditoria de consultas:** Devolve uma lista de todas as consultas realizadas ao sistema, indicando o autor da consulta, a localidade e a data/hora.
- D. **Estimar previsão futura:** Com base num número especificado de dias de dados históricos, o sistema estima a condição climática mais provável para uma localidade. A lógica de estimação será a moda estatística (a condição mais frequente no período).

3 Requisitos Não Funcionais

O projeto tem ainda os seguintes requisitos:

- A aplicação não necessita de um sistema de autenticação nesta fase mas é necessário registar o autor de cada consulta.
- Toda a informação deverá ser armazenada numa base de dados relacional (ex: PostgreSQL).
- A plataforma deverá ser implementada em **Java (versão 17 ou superior)**, sem o uso de frameworks de aplicação como o Spring Boot. A camada de dados será implementada utilizando **JDBC (Java Database Connectivity)**.
- O projeto deve ser dividido em camadas. Não há uma camada de apresentação. A camada de negócios deverá seguir o padrão arquitetural **Transaction Script**. Cada caso de uso corresponderá a um procedimento no código. A camada de acesso aos dados será implementada utilizando o padrão **Row Data Gateway**.
- O repositório deve aceitar apenas código com o nível de qualidade aceite pela equipa (ex.: *pre-commit*, testes unitários).
- O projeto correrá num ambiente Docker (será a forma de deploy no servidor de produção).
- É **obrigatório** o uso de *testes unitários* e *testes de integração* para garantir a qualidade do código. A cobertura mínima de testes deverá ser de 80% do código.
- A base de dados deve ser populada com dados de teste para permitir a validação do funcionamento do sistema. Estes dados devem ser inseridos através de um script SQL ('data.sql').
- O controlo da participação de cada membro do grupo será feito via atividade no repositório git, com base no número e qualidade dos *commits*.
- As decisões técnicas devem ser justificadas num relatório técnico.

4 Tarefas da Fase 1

A equipa deverá:

- Fazer fork do repositório original (<https://git.alunos.di.fc.ul.pt/css000/weatherwise>), para a conta de um dos elementos do grupo.
- Dar acesso a todos os membros do grupo, como Maintainer.

- Dar acesso à conta `css000`, como Reporter.
- Criar uma pasta *docs*, onde se colocará um único documento PDF com todos os diagramas e o relatório.
- Desenhar o **diagrama Entidade-Relação (ER)** para a base de dados, tendo em conta todos os casos de uso.
- Desenhar o diagrama de sequência de sistema (SSD) para o caso de uso **D (Estimar Previsão Futura)**.
- Desenhar um esboço do diagrama de classes que mostre a divisão em camadas (ex: Camada de Serviço, Camada de Negócios, Camada de Acesso a Dados).
- Criar um script SQL (`'schema.sql'`) que cria todas as tabelas, chaves primárias/estrangeiras e restrições necessárias.
- Justificar no relatório as decisões tomadas no desenho do esquema da base de dados (ex: escolha de tipos de dados, chaves, índices).
- Incluir no relatório as garantias que o sistema oferece relativamente à consistência dos dados.
- Implementar testes unitários e de integração para garantir que a lógica de negócio está correta.

Para validar o funcionamento da aplicação, é fundamental que o sistema seja capaz de responder a questões pertinentes sobre os dados armazenados. A seguir, apresenta-se um conjunto de exemplos de perguntas que demonstram as funcionalidades esperadas:

1. Qual é a previsão do tempo para “Lisboa” hoje?
2. Quantas vezes a previsão para o “Porto” já foi consultada?
3. Qual utilizador fez mais consultas para a localidade “Braga”?
4. Qual a previsão estimada para “Faro” com base nos dados dos últimos 15 dias?
5. Qual foi a condição climática mais frequente em “Coimbra” no último mês?

5 Como Entregar

Para entregar o trabalho, basta criar uma tag chamada *fase1* e enviá-la para o repositório. O repositório deverá conter o código-fonte do projeto, os ficheiros necessários para o correr num ambiente Docker (Dockerfile, etc.), bem como um único documento PDF na pasta *docs* com o relatório e todos os diagramas.

```
git tag fase1
git push origin fase1
```

Deve-se confirmar que o vosso projeto está acessível à conta `css000` na tag *fase1*. Caso contrário, terão 0 nesta entrega. É importante garantir que o ambiente de execução possa ser reproduzido pela equipa docente sem erros de compilação ou outros que impeçam a sua execução. O projeto **obrigatoriamente** deverá estar num ambiente Docker. Falhas em **EXECUTAR** o projeto, num ambiente Docker, pela equipa docente, resultarão em **grave penalização nesta entrega**. Falha em apresentar testes unitários e de integração, bem como a cobertura mínima de 80% do código, também resultará em **grave penalização nesta entrega**.

6 Critérios de Avaliação

Esta entrega corresponde a 25% da nota final do projeto. Para perceberem onde se devem focar, segue uma lista de critérios que serão avaliados:

6.1 Processo de Desenvolvimento

- Granularidade e frequência dos *commits*. Todos os membros do grupo devem contribuir de forma equitativa. *Disparidades significativas serão penalizadas*.
- Qualidade das mensagens de *commit*.
- Qualidade do código presente no repositório (organização, clareza, boas práticas).
- Funcionalidade e correção dos casos de uso implementados.

6.2 Requisitos

- Em que medida os requisitos funcionais e não funcionais são abrangidos pela modelação e implementação.

6.3 Testes

- Cobertura dos casos de uso pelos testes escritos.
- Cobertura de casos de erro e excecionais pelos testes.

6.4 Relatório e Diagramas

- Descrição clara da arquitetura em camadas.
- Qualidade do Diagrama Entidade-Relação e do Diagrama de Classes.
- Qualidade do Diagrama de Sequência de Sistema (SSD).
- Qualidade e correção do esquema da base de dados e do script SQL.
- Qualidade da justificação das decisões de desenho da base de dados.
- Qualidade da implementação da lógica de negócio nos Transaction Scripts.

Nota Final: Em caso de dúvidas, utilizem o fórum de discussão na plataforma académica. Boa sorte com o desenvolvimento do *WeatherWise*!